RELAZIONE TECNICA

Consolidamento di Archivi Fotografici Multi-Sorgente con Resilienza ai Blackout

Progettazione, sviluppo e correzione di un sistema per deduplicare e riunire un archivio fotografico distribuito su dischi multipli quasi identici, inclusi i contenuti sepolti in migliaia di archivi di backup annidati

36.543	11M+	9
FOTO/VIDEO UNICI RECUPERATI	FILE ESAMINATI	BUG CRITICI CORRETTI

photo-consolidator · Progetto open source (Licenza MIT) · 2026

INDICE

Sommario dei contenuti

1. Introduzione e contesto del problema

1.1 Il problema del consolidamento multi-sorgente

1.2 Perché la resilienza ai blackout è un requisito, non un'opzione

1.3 Scala del caso reale affrontato

2. Architettura della soluzione

2.1 SQLite come stato persistente centrale

2.2 Il modello dati

2.3 Le fasi del processo (Fase 0 – Fase 8)

2.4 Deduplica per contenuto

2.5 Gestione degli archivi annidati

3. Parallelizzazione e prestazioni

4. Catalogo dei bug scoperti e delle correzioni applicate

4.1 – 4.9 Nove bug critici, causa, sintomo, correzione

5. Avvertenze per chi riusa o estende il codice

6. Pubblicare il progetto come software libero

7. Conclusioni e lavori futuri

CAPITOLO 1

Introduzione e contesto del problema

1.1 Il problema del consolidamento multi-sorgente

Chiunque abbia accumulato fotografie digitali per più di un decennio conosce bene il problema: copie di sicurezza fatte in momenti diversi, su dischi diversi, spesso senza un criterio coerente, finiscono per produrre **tre, quattro, cinque copie quasi identiche** dello stesso archivio fotografico. "Quasi identiche" è la parola chiave: non sono copie esatte, perché nel tempo si aggiungono foto nuove a un disco e non all'altro, si eliminano duplicati manualmente in modo incoerente, si eseguono backup automatici di sistema operativo (Windows, in particolare) che finiscono per includere fotografie dentro cartelle di archiviazione compresse.

Il risultato è un disordine multi-dimensionale: la stessa fotografia può trovarsi come file diretto su un disco, dentro uno zip di backup su un secondo disco, e dentro un archivio RAR multivolume nidificato dentro un altro zip su un terzo disco. Un consolidamento manuale è semplicemente impraticabile oltre poche migliaia di file; oltre il milione di file, come nel caso reale descritto in questa relazione, è necessario un sistema automatizzato, affidabile e capace di lavorare per giorni senza supervisione continua.

1.2 Perché la resilienza ai blackout è un requisito, non un'opzione

Un processo che scansiona, estrae ed elabora milioni di file su dischi USB può richiedere **giorni** di esecuzione continua. In questo lasso di tempo sono praticamente certi: interruzioni di corrente, riavvii del sistema, necessità di fermare temporaneamente il processo per liberare risorse, crash imprevisti causati da singoli file corrotti o casi limite non previsti.

> **PRINCIPIO DI PROGETTAZIONE FONDAMENTALE**
>
> Se un processo di questa durata non può essere interrotto e ripreso senza perdita di lavoro, è *di fatto* inutilizzabile nella pratica. La resilienza ai blackout non è una funzionalità accessoria: è il requisito architetturale che determina ogni altra scelta di design descritta in questa relazione.

La soluzione adottata consiste nel trattare un database SQLite come **unica fonte di verità** sullo stato di avanzamento, con ogni singola operazione (scansione di un file, estrazione di un archivio, hash calcolato, decisione di deduplica, copia fisica) registrata in modo atomico e durevole (`PRAGMA synchronous=FULL`) prima di proseguire. Il programma può essere interrotto in un punto qualsiasi — anche con `kill -9` o un distacco di corrente reale — e ripreso semplicemente

rilanciando lo stesso comando: riprenderà esattamente da dove si era fermato, senza ripetere lavoro già svolto né, soprattutto, senza perdere silenziosamente dei file.

1.3 Scala del caso reale affrontato

Questa relazione documenta lo sviluppo iterativo del programma nel contesto di un caso reale:

Dimensione	Valore
Dischi sorgente	3 dischi USB da 3,6–13 TB, quasi duplicati fra loro
File totali esaminati	oltre 11.000.000 (inclusi i contenuti degli archivi estratti)
Archivi di backup processati	oltre 59.000 file zip/7z/rar/tar, molti annidati
Durata complessiva del progetto	circa 24 ore di elaborazione effettiva, su più sessioni
Risultato finale	36.543 foto/video unici, 411 GB, copiati e verificati con hash

Durante questo lavoro sono stati scoperti e corretti **nove bug distinti**, alcuni dei quali avrebbero causato una perdita silenziosa di fotografie reali se non identificati: sono catalogati in dettaglio nel Capitolo 4, poiché costituiscono probabilmente la parte di maggior valore pratico di questa relazione per chiunque affronti un problema simile.

CAPITOLO 2

Architettura della soluzione

2.1 SQLite come stato persistente centrale

La scelta di SQLite (anziché, ad esempio, tenere lo stato in memoria con checkpoint periodici su file, o usare un database client-server) è deliberata: SQLite offre transazioni ACID, un singolo file autocontenuto facile da ispezionare e sottoporre a backup, e prestazioni più che sufficienti per il carico di lavoro (scritture sequenziali da un unico thread scrittore, letture occasionali concorrenti). La configurazione chiave è:

```
conn.execute("PRAGMA journal_mode=WAL;")
conn.execute("PRAGMA synchronous=FULL;")   # <- la garanzia anti-blackout
conn.execute("PRAGMA busy_timeout=30000;")
```

`synchronous=FULL` è la scelta più importante e volutamente non ottimizzata per la velocità: garantisce che ogni transazione sia fisicamente scritta su disco (fsync) prima che SQLite ritorni il controllo al programma, cosa che rende impossibile la corruzione del database in caso di interruzione improvvisa dell'alimentazione. Il costo in prestazioni è accettabile perché le scritture sono raggruppate in batch (commit ogni N operazioni, non una per una) nei punti del codice dove il throughput è critico.

2.2 Il modello dati

Lo schema è composto da quattro tabelle:

```
files(id, full_path, source_root, size, mtime, category, status,
      sha256, width, height, exif_date, target_path,
      origin_archive_id, cache_path, error_msg, updated_at)

archives(id, full_path, source_root, size, status, scratch_disk,
         extract_dir, depth, parent_archive_id, error_msg, updated_at)

hashes(sha256 PRIMARY KEY, target_path, first_file_id)

scan_complete(source_root PRIMARY KEY, completed_at)
```

Ogni file transita attraverso una macchina a stati esplicita: `indexed` → `hashed` → (`duplicate` | `would_copy` / `copied` | `too_small` | `error`). Gli archivi seguono un percorso analogo: `pending` → `extracting` → `extracted` → `cleaned` (o `error` / `skipped`). Questa

esplicitazione dello stato è ciò che rende possibile la ripresa: al riavvio, il programma interroga semplicemente "quali righe non sono ancora in uno stato terminale?" e riprende da lì.

2.3 Le fasi del processo

Fase	Compito
0	Verifica spazio disco disponibile e raggiungibilità delle fonti configurate
1	Scansione parallela del filesystem di ogni fonte (un thread per fonte)
2	Estrazione parallela degli archivi di backup, con gestione ricorsiva degli annidati
3–4	Per ogni file media: lettura dimensioni/data (exiftool) e calcolo hash SHA-256
5–6	Deduplica (confronto hash) e copia fisica con verifica post-copia
7	Report finale aggregato (conteggi per stato, per fonte, errori)
8	Pulizia delle cartelle di scratch residue

2.4 Deduplica per contenuto

La deduplica non si basa mai su nome file o dimensione, ma esclusivamente sull'hash SHA-256 del contenuto completo. Il primo file con un dato hash che viene incontrato determina il percorso di destinazione (organizzato per data EXIF, `AAAA/MM/GG`, con fallback su una cartella `senza-data` quando l'EXIF manca); ogni file successivo con lo stesso hash viene marcato `duplicate` e punta allo stesso target, senza essere fisicamente ricopiato. La tabella `hashes` funge da indice univoco per questa decisione ed è protetta da un vincolo `PRIMARY KEY` sull'hash.

2.5 Gestione degli archivi annidati

È comune che un archivio di backup contenga a sua volta altri archivi (un backup settimanale che include l'intero profilo utente, a sua volta contenente vecchi zip di foto). Il programma supporta un livello configurabile di annidamento (`MAX_ARCHIVE_DEPTH`): quando un archivio annidato viene scoperto durante l'estrazione del genitore, viene elaborato **immediatamente**, prima che la cartella di estrazione del genitore (da cui l'annidato dipende fisicamente) venga ripulita. Il perché di questa scelta, non ovvia, è raccontato nel dettaglio al Bug 4.7.

CAPITOLO 3

Parallelizzazione e prestazioni

La prima versione funzionante del programma elaborava gli archivi in modo strettamente sequenziale: estrai, indicizza, hash, copia, pulisci, passa al successivo. Misurando l'utilizzo delle risorse durante l'esecuzione reale è emerso un dato sorprendente: **il processo utilizzava solo il 20% di un singolo core**, su una macchina con 12 core disponibili e dischi quasi completamente inattivi. Il collo di bottiglia non era hardware, ma l'assenza di sovrapposizione fra l'attesa I/O di un'estrazione e la successiva.

Architettura produttore/consumatore per l'estrazione

La soluzione adottata separa nettamente due responsabilità:

- **Thread worker (pool):** eseguono esclusivamente l'estrazione (chiamata a `subprocess`, che rilascia il GIL durante l'attesa I/O) e non toccano mai il database.
- **Thread principale (unico scrittore):** consuma i risultati da una coda e compie tutte le operazioni che toccano SQLite — indicizzazione, hashing, deduplica, copia — mantenendo la garanzia di scrittore singolo che SQLite richiede per l'integrità.

La concorrenza di estrazione è inoltre limitata **per disco fisico sorgente**: un solo thread alla volta per ciascun disco rotante (HDD), perché leggere più flussi contemporaneamente da un disco con testine meccaniche non aumenta il throughput — lo degrada, a causa del seek thrashing. Per il contenuto già estratto su scratch locale (SSD/NVMe), dove il seek non è un problema, la concorrenza è invece più alta.

> **RISULTATO MISURATO**
>
> Il throughput di elaborazione è passato da circa 60–90 MB/s a oltre 100 MB/s sostenuti, con picchi superiori, rappresentando un guadagno di velocità di circa 2–3× rispetto alla pipeline sequenziale originale, a parità di hardware.

CAPITOLO 4

Catalogo dei bug scoperti e delle correzioni

Questo capitolo documenta, in ordine cronologico di scoperta, i nove problemi più significativi identificati durante lo sviluppo e la conduzione del caso reale. Ognuno segue lo stesso schema: **sintomo osservato**, **causa radice**, **correzione applicata**. Sono inclusi perché rappresentano insidie non ovvie che chiunque affronti un problema simile incontrerebbe quasi certamente.

4.1 — Scratch exFAT/NTFS incompatibile con percorsi Windows

SINTOMO

Migliaia di archivi di backup fallivano l'estrazione con errori `cannot create` / `errno=22`, apparentemente casuali.

CAUSA RADICE

Molti backup Windows salvano intere voci come `C:\Users\...\file.ext` come nome singolo in uno zip. `unzip` le interpreta come percorso con separatori, tentando di creare una cartella letteralmente chiamata `C:` — impossibile su exFAT/NTFS, dove `:` è un carattere riservato.

CORREZIONE

Lo scratch di estrazione primario è stato spostato su un filesystem nativo Linux (ext4), che non impone alcuna restrizione sui byte ammessi in un nome file a parte `/` e NUL.

4.2 — Crash UnicodeEncodeError su nomi file con encoding legacy

SINTOMO

Il processo terminava con `UnicodeEncodeError` non gestita, interrompendo l'intera esecuzione.

CAUSA RADICE

Conseguenza collaterale della correzione 4.1: ext4 accetta qualsiasi sequenza di byte come nome file, inclusi nomi codificati in code page storiche (CP850, tipica di vecchi zip DOS/Windows per lettere accentate). Salvare quel percorso in SQLite (che richiede UTF-8 valido) falliva.

CORREZIONE

Funzione dedicata che rileva nomi non validi in UTF-8, recupera i byte originali (tramite `surrogateescape`) e li ri-decodifica esplicitamente come CP850, rinominando il file/cartella sul disco con il nome corretto.

4.3 — File "da copiare" persi alla pulizia dello scratch

SINTOMO

5.570 file già identificati come unici da conservare (durante una simulazione "dry-run") risultavano irraggiungibili al momento della copia reale.

CAUSA RADICE

La cartella di estrazione temporanea di un archivio viene ripulita subito dopo la sua elaborazione, **anche in modalità simulazione**: per i file provenienti dall'interno di un archivio, quella cancellazione avveniva prima che il file fosse mai stato effettivamente copiato, dato che la simulazione per definizione non scrive sulla destinazione finale.

CORREZIONE

Introduzione di una cache persistente separata (su filesystem di sistema, non scratch effimero): quando un file viene identificato come "da conservare" durante una simulazione, se la sua fonte è una cartella di estrazione temporanea, ne viene salvata subito una copia nella cache, che sopravvive alla pulizia. I 5.570 file già persi sono stati recuperati ri-estraendo i rispettivi archivi originali (ancora presenti, invariati, sui dischi sorgente).

4.4 — Distruzione della parallelismo per i dischi rotanti

SINTOMO

Utilizzo CPU al 20% su un sistema a 12 core, con dischi quasi inattivi, nonostante decine di migliaia di archivi ancora da elaborare.

CAUSA RADICE

Pipeline di estrazione strettamente sequenziale: nessuna sovrapposizione fra l'attesa I/O di un'estrazione e la successiva.

CORREZIONE

Architettura produttore/consumatore descritta nel Capitolo 3, con parallelismo limitato per disco fisico sorgente per evitare il seek thrashing sugli HDD rotanti.

4.5 — Falso reporting dello stato "extracting"

SINTOMO

Dopo l'introduzione della parallelizzazione, il conteggio di archivi "in coda" crollava artificialmente da decine di migliaia a poche centinaia nell'istante stesso del riavvio, senza che fosse stato svolto lavoro reale.

CAUSA RADICE

Un tentativo iniziale di marcare in blocco tutti gli archivi in coda come "in estrazione" nel momento in cui venivano sottomessi al pool di thread, prima ancora che un worker li prendesse effettivamente in carico.

CORREZIONE

Rimossa la marcatura preventiva: un archivio resta "in coda" (`pending`) finché un worker non lo elabora davvero, mantenendo le statistiche di avanzamento accurate senza alcun impatto sulla logica di ripresa.

4.6 — Cache degli archivi non ripulita dopo l'esclusione di cartelle spazzatura

SINTOMO

Un'esclusione aggiunta successivamente (es. cartelle cache del browser) non aveva alcun effetto su decine di migliaia di archivi già registrati in una scansione precedente, poiché l'esclusione si applica solo durante la scansione del filesystem.

CAUSA RADICE

Le esclusioni per nome cartella (`EXCLUDE_DIR_NAMES`) vengono valutate solo mentre si cammina l'albero delle directory; non hanno effetto retroattivo su righe già inserite in una sessione precedente all'introduzione della regola.

CORREZIONE

Script di pulizia mirato che ricontrolla ogni percorso già registrato contro l'insieme aggiornato di esclusioni e aggiorna lo stato di conseguenza, oltre alla correzione preventiva nel codice.

4.7 — Archivi annidati orfani per sempre

SINTOMO

53.370 archivi annidati risultavano permanentemente bloccati in stato "in coda", falliti ad ogni tentativo con "fonte non raggiungibile", indipendentemente da quante volte il processo venisse riavviato.

CAUSA RADICE

Un archivio annidato, scoperto durante l'elaborazione del genitore, veniva solo *registrato* nel database per essere elaborato alla **prossima esecuzione** del programma. Ma la cartella di estrazione del genitore (da cui l'annidato dipende fisicamente, essendo un file al suo interno) veniva ripulita alla fine della stessa esecuzione corrente — rendendo l'annidato irraggiungibile per sempre a ogni tentativo futuro.

CORREZIONE

Gli archivi annidati vengono ora elaborati **ricorsivamente e immediatamente**, nella stessa sessione del genitore, prima che la sua cartella di estrazione venga ripulita. I 53.370 archivi già bloccati sono stati recuperati identificando i rispettivi 5.140 archivi genitori (ancora integri sui dischi sorgente) e ri-processandoli con la correzione attiva.

4.8 — Archivi copiati integralmente come se fossero singole foto

SINTOMO

Durante la copia reale, il processo tentava di copiare un file da 26,9 GB — risultato essere un intero archivio ZIP di esportazione Google Takeout, non una singola fotografia. Verifica estesa: 5.138 file archivio (402,8 GB) erano nello stesso stato.

CAUSA RADICE

Stato residuo incoerente nel database, risalente a una versione precedente del codice (prima dell'introduzione delle correzioni precedenti): la riga `files` corrispondente al file-archivio stesso era rimasta erroneamente nello stato "hashed"/"da copiare" invece di essere marcata come "archiviata" al termine della sua effettiva estrazione, che nel frattempo *era* stata completata correttamente (verificato confrontando la dimensione totale del contenuto estratto con quella dell'archivio compresso: coincidevano).

CORREZIONE

Sincronizzazione dello stato di ogni file-archivio con lo stato reale del suo record di elaborazione, senza necessità di ri-estrazione (il lavoro reale era già stato svolto correttamente). Aggiunta una regola di non regressione: un file di categoria "archivio" non deve mai transitare nella pipeline di hashing/copia riservata ai contenuti media.

4.9 — Codifica CP850 non sufficiente per tutti i casi

SINTOMO

Un sottoinsieme di archivi continuava a fallire l'estrazione con l'errore "cannot create" anche dopo la correzione 4.1, con messaggi contenenti sequenze di sostituzione (`?`) al posto di lettere accentate.

CAUSA RADICE

`unzip` assume UTF-8 di default su Linux; per gli archivi con nomi in una code page storica diversa da quella assunta, la decodifica produce byte non validi per il filesystem.

CORREZIONE

Strategia a cascata: tentativo con encoding UTF-8 di default, poi ritentativo esplicito con code page CP850, infine fallback su `7z` (che gestisce autonomamente il charset OEM ed è generalmente più tollerante). Solo se tutti e tre falliscono l'archivio è marcato definitivamente in errore.

CAPITOLO 5

Avvertenze per chi riusa o estende il codice

Chi intende adattare questo programma a un caso d'uso diverso dovrebbe prestare particolare attenzione ai seguenti punti, ciascuno collegato a un bug reale descritto nel Capitolo 4:

1. **Non spostare lo scratch di estrazione su un filesystem exFAT/NTFS/FAT32.** Deve sempre essere un filesystem nativo Linux, altrimenti si ripresenteranno gli errori del Bug 4.1.
2. **Non rimuovere la cache persistente per i file provenienti da archivi** (`cache_ephemeral_source`). Senza di essa si ripresenta silenziosamente il Bug 4.3, con perdita di fotografie senza alcun messaggio di errore visibile.
3. **Non processare gli archivi annidati in un secondo momento rispetto al genitore.** Devono essere gestiti ricorsivamente e immediatamente (Bug 4.7), altrimenti diventano irraggiungibili per sempre alla prima pulizia dello scratch.
4. **Un file di categoria "archivio" non deve mai entrare nella pipeline di hash/copia dei media.** Va sempre e solo estratto; il suo contenuto va deduplicato individualmente (Bug 4.8).
5. **Testare sempre prima con** `--dry-run` su un sottoinsieme di dati prima di un'esecuzione reale su un archivio di grandi dimensioni, e leggere attentamente il report finale.
6. **Mantenere** `PRAGMA synchronous=FULL`**.** È la garanzia fondamentale di resilienza ai blackout; disattivarla per guadagnare velocità espone al rischio di corruzione del database in caso di interruzione improvvisa.
7. **Verificare sempre lo spazio libero** sui dischi di destinazione e di scratch prima di un'esecuzione di grandi dimensioni; il sistema di fallback multi-disco aiuta ma non sostituisce una pianificazione dello spazio.

CAPITOLO 6

Pubblicare il progetto come software libero

Il codice è concesso con licenza **MIT** (il file `LICENSE` è incluso nel progetto): permette a chiunque di usarlo, modificarlo e ridistribuirlo liberamente, anche a fini commerciali, senza garanzia. Per pubblicarlo pubblicamente sono disponibili diverse strade, descritte di seguito in ordine di semplicità.

6.1 Creare un account GitHub (consigliato)

Non esiste un modo per pubblicare *su GitHub* senza un account GitHub: è un passaggio inevitabile, ma gratuito e rapido (circa due minuti):

1. Vai su **github.com** e scegli "Sign up"
2. Serve solo un indirizzo email, una password e un nome utente: nessun documento, nessun pagamento
3. Conferma l'email tramite il link che ricevi
4. Crea un nuovo repository pubblico (es. `photo-consolidator`)
5. Dalla cartella del progetto: `git remote add origin https://github.com/TUO-UTENTE/photo-consolidator.git`
6. `git push -u origin master` (ti verrà chiesta l'autenticazione: GitHub richiede un *personal access token* al posto della password, generabile dalle impostazioni dell'account in Settings → Developer settings → Personal access tokens)

6.2 Alternative che non richiedono un account Microsoft

Dal 2018 GitHub è di proprietà Microsoft; se questo è il motivo della resistenza a crearne uno, esistono alternative equivalenti, tutte gratuite per progetti open source:

Piattaforma	Note
GitLab.com	Funzionalmente molto simile a GitHub, azienda indipendente (con sede negli USA); account gratuito con solo email
Codeberg.org	No-profit europea (Germania), basata su software libero (Forgejo); nessun legame con grandi aziende tech
SourceHut (sr.ht)	Minimalista, orientato a sviluppatori esperti, filosofia radicalmente open source

La procedura di pubblicazione è concettualmente identica su tutte: creare un account, creare un repository vuoto, collegare il repository locale già pronto in `/home/titoio/photo-consolidator` (già inizializzato con `git init` e un primo commit) con `git remote add origin <url>`, quindi `git push`.

> **STATO ATTUALE DEL PROGETTO**
>
> Il repository Git locale è già pronto in `~/photo-consolidator/`, con il primo commit effettuato. Manca solo la scelta della piattaforma e la creazione dell'account per completare la pubblicazione con `git push`.

CAPITOLO 7

Conclusioni e lavori futuri

Il caso reale descritto in questa relazione dimostra come un problema apparentemente semplice — "unisci queste foto sparse su più dischi" — nasconda in realtà una notevole complessità quando affrontato alla scala di milioni di file e centinaia di gigabyte, distribuiti su hardware eterogeneo (dischi USB rotanti, SSD, filesystem diversi) e accumulati nel tempo senza una struttura coerente. La maggior parte dei bug scoperti non riguardava la logica di business (identificare foto uniche) ma le **interazioni ai bordi del sistema**: filesystem con regole diverse, processi interrotti a metà, stati intermedi mai completamente terminali.

Possibili direzioni di miglioramento future, non implementate in questa versione:

- Parallelizzazione anche della fase di hashing/copia con scritture concorrenti su SQLite (attualmente il collo di bottiglia residuo, dopo la parallelizzazione dell'estrazione)
- Rilevamento di near-duplicate (foto quasi identiche ma non byte-per-byte uguali, es. la stessa foto ricompressa con qualità diversa)
- Interfaccia web opzionale per monitorare l'avanzamento senza dover interrogare il database o i log direttamente
- Supporto nativo per fonti di rete (SMB/NFS) con throttling automatico invece che configurato manualmente

---

Relazione tecnica generata a corredo del progetto open source **photo-consolidator** (licenza MIT). Documento prodotto con l'assistenza di Warp (agent@warp.dev).